Process Management in Linux

Process management is one of the most important responsibilities of the Linux operating system. It involves creating, scheduling, monitoring, and terminating processes to ensure efficient use of system resources like CPU and memory.

1. What is a Process in Linux?

A **process** is an instance of a program that is currently running.

When you execute a program, Linux creates a process for that program and allocates system resources such as:

- CPU time
- Memory
- File descriptors
- I/O resources

Example

If you run:

firefox

Linux creates a **process** for the Firefox program.

Each process has a unique identifier called **PID (Process ID)**.

---

2. Process Lifecycle (States of a Process)

A process goes through several states during its lifetime.

Common Process States

State	Description

New	Process is being created
Ready	Process is waiting to be assigned CPU
Running	Process is currently executing
Waiting / Sleeping	Process is waiting for an event or I/O
Stopped	Process execution is paused
Zombie	Process finished but entry still exists in process table
Terminated	Process is completed and removed

View Process States
ps aux

Output example:

USER PID %CPU %MEM STAT COMMAND
root 1020 0.1 0.2 S sshd

Where `STAT` indicates the state:

- **R** → Running
- **S** → Sleeping
- **T** → Stopped
- **Z** → Zombie

---

3. Types of Processes in Linux

3.1 Foreground Process

Runs in the terminal and interacts with the user.

Example:

nano file.txt

Terminal will be occupied until process finishes.

3.2 Background Process

Runs without blocking the terminal.

sleep 100 &

The & symbol runs the process in background.

3.3 Daemon Process

Background services that run continuously.

Examples:

- sshd
- cron
- httpd
- systemd

List daemon processes:

ps -ef | grep daemon

---

4. Process Identification in Linux

Every process has unique identifiers.

Identifier	Description
PID	Process ID
PPID	Parent Process ID

UID	User ID
GID	Group ID

Example
ps -ef

Output:

```
UID   PID  PPID  CMD
root  1200 1     nginx
```

Here:

- PID → 1200
- Parent process → 1 (init/systemd)

---

5. Parent and Child Processes

Linux processes follow a **hierarchical structure**.

- A process that creates another process is called the **parent process**
- The new process is called a **child process**

Example:

```
systemd (PID 1)
  └── sshd
      └── bash
          └── top
```

Display process tree:

pstree

---

6. Process Scheduling

Process scheduling is the **method Linux uses to decide which process should use the CPU and for how long**.

Since many processes run at the same time but the CPU can execute **only one process at a time (per core)**, the Linux kernel schedules them efficiently.

Think of it like **people waiting in line to use a printer** — only one person can print at a time, so the system decides whose turn comes next.

6.1 Why Process Scheduling is Needed

A computer runs many processes simultaneously such as:

- Web browser
- Terminal
- System services
- Background tasks

Example:

firefox

nginx

mysql

backup.sh

If Linux allowed one process to run forever, other programs would **never get CPU time**.

So Linux uses a **scheduler** to distribute CPU time among processes.

6.2 Simple Real-World Example

Imagine a **teacher asking students questions in class**.

Students = Processes
Teacher = CPU Scheduler

The teacher gives each student **a small amount of time to answer**, then moves to the next student.

Example:

Student A → 1 second

Student B → 1 second

Student C → 1 second

Student D → 1 second

Then the cycle repeats.

This is how Linux shares the CPU between processes.

6.3 CPU Time Slice (Time Quantum)

Linux gives each process a **small time slot** called a **time slice**.

Example:

Process	CPU Time
Process A	10 ms
Process B	10 ms
Process C	10 ms

After the time expires, Linux switches to another process.

This is called **context switching**.

6.4 Context Switching

When CPU switches from one process to another, it is called **context switching**.

Example flow:

Process A running

↓

time slice finished

↓

Process B running

↓

time slice finished

↓

Process C running

Linux saves the state of the previous process and loads the next process.

6.5. Process Scheduling Types

6.5.1 Preemptive Scheduling (Used by Linux)

In this method, the operating system **can interrupt a running process** and give CPU to another process.

Example:

Process A running

High priority process B arrives

Linux stops A and runs B

This ensures important processes run faster.

6.5.2 Non-Preemptive Scheduling

In this method, a process runs **until it finishes or releases CPU**.

Example:

Process A starts

Process A runs until completion

Then Process B runs

This approach is rarely used in modern Linux systems.

---

7. Viewing Running Processes

7.1 ps Command

Displays currently running processes.

ps

Detailed view:

ps -ef

Another format:

ps aux

7.2 top Command

Shows **real-time system processes**.

top

Displays:

- CPU usage

- Memory usage
- Process list
- Load average

7.3 htop Command

Improved version of `top`.

htop

Install if not available:

sudo yum install htop

Features:

- Color interface
- Easy navigation
- Kill processes interactively

7.4 pstree Command

Shows process hierarchy in tree format.

pstree

Example output:

systemd─┬─sshd───bash───top

└─cron

7.5 pidof Command

Finds PID of a process.

pidof nginx

Example output:

2356

7.6 pgrep Command

Search process by name.

pgrep apache2

---

8. Controlling Processes

8.1 kill Command

Terminates a process using its PID.

kill PID

Example:

kill 1234

8.2 Force Kill Process

kill -9 PID

Example:

kill -9 1234

This sends **SIGKILL signal**.

8.3 killall Command

Kills processes by name.

killall firefox

8.4 pkill Command

Kills process by pattern.

pkill nginx

---

9. Signals in Linux

Signals are messages sent to processes to control them.

Signal	Number	Purpose
SIGTERM	15	Graceful termination
SIGKILL	9	Force kill
SIGSTOP	19	Pause process
SIGCONT	18	Resume process

Example:

kill -9 1234

This forcefully kills process **1234**.

---

10. Job Control in Linux

Job control allows managing processes in the terminal.

10.1 Run a process in background

sleep 200 &

10.2 List background jobs

jobs

10.3 Bring job to foreground

fg %1

10.4 Send job to background

bg %1

---

11. Process Priority (Nice Value)

Linux assigns priority to processes using **nice values**.

Range:

-20 (Highest Priority)

0 (Default)

19 (Lowest Priority)

Check priority:

top

11.1 Run process with priority

nice -n 10 command

Example:

nice -n 10 tar -czf backup.tar.gz /data

11.2 Change priority of running process

renice -n 5 -p PID

Example:

renice -n 5 -p 2345

---

12. Monitoring System Processes

12.1 uptime command

Shows system load.

uptime

Example output:

10:30:12 up 2 days, load average: 0.10, 0.20, 0.30

12.2 watch command

Runs a command repeatedly.

Example:

watch -n 2 ps

This runs `ps` every 2 seconds.

---

13. Zombie Processes

A **zombie process** is a process that has completed execution but still exists in the process table.

Reason:
Parent process has not read the exit status.

Identify zombies:

ps aux | grep Z

Example output:

Z 1234 zombie_process

---

14. Orphan Processes

An **orphan process** occurs when the parent process terminates before the child.

In Linux, orphan processes are adopted by **systemd (PID 1)**.

---

15. Important Linux Process Management Commands (Summary)

Command	Purpose
ps	Display processes
top	Real-time process monitoring
htop	Advanced process viewer
pstree	Show process tree
pidof	Find PID
pgrep	Search process
kill	Terminate process
killall	Kill by name
pkill	Kill by pattern
jobs	List background jobs
bg	Resume job in background
fg	Bring job to foreground
nice	Start process with priority

renice	Change process priority
uptime	Show system load
watch	Run command repeatedly

---

16. Real-World Examples of Linux Process Management

Understanding process management becomes easier when you see how it is used in **real production environments** such as web servers, databases, DevOps pipelines, and system administration tasks.

Below are some **practical real-world scenarios** where Linux process management commands are used.

16.1 Monitoring High CPU Usage on a Production Server

Scenario

A website hosted on a Linux server becomes **slow**. As a system administrator, you need to identify which process is consuming the CPU.

Step 1: Check running processes

top

Output example:

```
PID USER   %CPU COMMAND
2345 root   90.5 java
1452 nginx   2.0 nginx
```

Here, the **Java process is consuming 90% CPU**.

Step 2: Investigate process

ps -fp 2345

Step 3: Restart or stop the process

kill 2345

Or force kill if needed:

kill -9 2345

This is a **common task in production servers**.

16.2 Running a Long Backup Process in Background

Scenario

A system administrator wants to create a **large backup**, but the process will take hours. The terminal should remain usable.

Step 1: Run backup in background

tar -czf backup.tar.gz /var/www &

The process runs in the **background**.

Step 2: Check background jobs

jobs

Output example:

[1]+ Running tar -czf backup.tar.gz /var/www &

Step 3: Check process using PID

ps aux | grep tar

16.3 Managing Web Server Processes

Scenario

A website hosted on **Nginx or Apache** stops responding.

Check running web server processes.

ps aux | grep nginx

Example output:

root 1010 nginx: master process

www-data 1011 nginx: worker process
www-data 1012 nginx: worker process

If needed, restart service:

systemctl restart nginx

Or kill stuck process:

killall nginx

16.4 Finding a Process by Name

Scenario

You want to know if **MySQL database** is running.

Use:

pgrep mysql

or

pidof mysql

Example output:

1325

Check detailed process:

ps -fp 1325

16.5 Controlling Foreground and Background Jobs

Scenario

You start a process accidentally that blocks your terminal.

Example:

sleep 500

Press:

CTRL + Z

Process stops.

Send it to background:

bg

Check job list:

jobs

Bring it back to foreground:

fg %1

16.6 Changing Process Priority for Critical Applications

Scenario

A **database server** should get more CPU priority than other processes.

Run with high priority:

nice -n -10 mysqld

Change priority of running process:

renice -n -5 -p 1325

This ensures **critical services run faster**.

16.7 Detecting Zombie Processes

Scenario

A misbehaving application creates zombie processes.

Check zombie processes:

ps aux | grep Z

Example output:

root 2034 0.0 0.0 Z [defunct]

Solution:

Restart parent process or reboot system.

16.8 Monitoring Processes Continuously

Scenario

A DevOps engineer wants to **monitor memory usage every 2 seconds**.

Use:

watch -n 2 free -m

Or monitor processes:

watch -n 2 ps aux

16.9 Viewing Process Hierarchy

Scenario

You want to understand which process started another process.

Use:

pstree

Example output:

```
systemd
├─sshd
│   └─bash
│       └─top
└─nginx
```

This shows **parent-child process relationships**.

16.10 Killing Multiple Processes at Once

Scenario

A developer accidentally started multiple **Python scripts**.

Kill all Python processes:

pkill python

or

killall python
